

Object Oriented Programming (Classes-V)

Abbas Khalid

April 21, 2025

Table of contents

- 1 Array of Objects
- 2 Pointers to Objects
- 3 Copying Object

Array of Objects

- We can declare an array of objects of a class just like we declare an array of any other type.
- Each element of an array of class objects has to be created individually and to do this, the compiler calls the default constructor for each element.
- If a class defines a parametrized constructor, we may initialize each object in an array by specifying an initialization list. However, the exact form of initialization list is decided by the number of parameters in the constructor.
- For objects, whose constructors have only one parameter, we can simply specify a list of initial values using the normal array-initialization syntax. As each element in the array is created, a value from the list is passed to the constructor's parameter.
- If an object's constructor requires two or more arguments, we will have to use the longer initialization form.

Array of Objects

① Array Declaration and Initialization Separately:

If we declare an array of objects and initialize it later, we indeed need a default constructor (either provided by the compiler or defined by the programmer). This is because when we declare an array without specifying initial values, the objects in the array are default-initialized, which requires a default constructor. If we don't provide a default constructor and the compiler doesn't generate one, we'll get a compilation error.

② Array Declaration and Initialization Together:

If we declare and initialize an array of objects at the same time, we can use a parameterized constructor to initialize the objects. When we initialize an array using an initializer list, we're providing values for each object directly, and these values can be passed to the parameterized constructor. In this case, we don't necessarily need a default constructor, as each object is initialized with specific values using the parameterized constructor.

Example 1

```

1 class Person {
2     int age;
3     public:
4         Person(){cout<<"Default\n";}
5         int getAge()const{return age;}
6         void setAge(int a){age = a;}
7 };
8 int main() {
9     const int size = 4; int age;
10    Person persons[size];
11    for(int i = 0; i<size; ++i){
12        cout<<"Enter age of Person("&<i+1<<"/"<<size<<"):";
13        cin>>age; persons[i].setAge(age);
14    }
15    for(int i = 0; i < size; ++i) {
16        cout<<"Age of Person("&<i+1<<"/"<<size<<"):";
17        cout<<persons[i].getAge() <<endl;
18    }
19    return 0;
20 }

```

```

Default
Default
Default
Default
Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12

```

Example 2

```
1 class Person {
2     int age;
3     public:
4         int getAge() const {return age;}
5         void setAge(int a) {age = a;}
6 };
7 int main() {
8     const int size = 4; int age;
9     Person persons[size];
10    for(int i = 0; i < size; ++i) {
11        cout << "Enter age of Person(" << i + 1 << "/" << size << "): ";
12        cin >> age; persons[i].setAge(age);
13    }
14    for(int i = 0; i < size; ++i) {
15        cout << "Age of Person(" << i + 1 << "/" << size << "): ";
16        cout << persons[i].getAge() << endl;
17    }
18    return 0;
19 }
```

Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12

Example 3

```
1 class Person {
2     int age;
3     public:
4         Person(){} // Required
5         Person(int a){age = a;} //Required
6         int getAge()const{return age;}
7 };
8 int main() {
9     const int size = 4; int age;
10    Person persons[size];
11    for(int i = 0; i<size; ++i){
12        cout<<"Enter age of Person("&<<i+1<<"/"<<size<<"):";
13        cin>>age; persons[i] = Person(age);
14    }
15    for(int i = 0; i < size; ++i) {
16        cout<<"Age of Person("&<<i+1<<"/"<<size<<"):";
17        cout<<persons[i].getAge()<<endl;
18    }
19    return 0;
20 }
```

```
Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12
```

Example 4

```

1 class Person {
2     int age;
3     public:
4         Person(){} //Not Required
5         Person(int a){age = a;} //Required
6         int getAge() const{return age;}
7 };
8
9 int main() {
10     const int size = 4;
11     Person P1(19);
12     Person P2(18);
13     Person P3(17);
14     Person P4(12);
15     Person persons[] = {P1,P2,P3,P4};
16     for(int i = 0; i < size; ++i) {
17         cout<<"Age of Person ("<<i+1<<"/"<<size<<"): ";
18         cout<<persons[i].getAge()<<endl;
19     }
20     return 0;
21 }

```

Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12

Example 5

```

1 class Person {
2     int age;
3     public:
4         Person(int a){age = a;}
5         int getAge()const{return age;}
6 };

```

Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12

```

7
8 int main() {
9     const int size = 4;

```

```

10
11     Person persons[] = {Person(19), Person(18), Person(17), Person(12)};

```

```

12     for(int i = 0; i < size; ++i) {
13         cout<<"Age of Person("<<i+1<<"/"<<size<<"):";
14         cout<<persons[i].getAge()<<endl;
15     }
16
17     return 0;
18 }

```

Example 6

```
1 class Person {  
2     int age;  
3     public:  
4         Person(int a){age = a;}  
5         int getAge() const{return age;}  
6 };
```

```
7  
8 int main() {  
9     const int size = 4;  
10    //ob = i changes to ob(i)  
11    Person persons[] = {19,18,17,12}; //for 1-parameter constructor only  
12    for(int i = 0; i < size; ++i) {  
13        cout<<"Age of Person ("<<i+1<<"/"<<size<<"):";  
14        cout<<persons[i].getAge()<<endl;  
15    }  
16  
17    return 0;  
18 }
```

Age of Person (1/4):19
Age of Person (2/4):18
Age of Person (3/4):17
Age of Person (4/4):12

Example 7

```
1 class Student {
2     int age;
3     double cgpa;
4 public:
5     Student(int age, double cgpa)
6         {this->age = age; this->cgpa = cgpa;}
7     int getAge() const {return age;}
8     double getCgpa() const {return cgpa;}
9 };
10
11 int main() {
12     const int size = 3;
13     //for a 2-parameter constructor, Compiler-dependent
14     Student s[] = { {19,3.9},{18,3.75},{17,3.0} };
15     for(int i = 0; i < size; ++i) {
16         cout<<"Student ("<<i+1<<"/"<<size<<"): ";
17         cout<<s[i].getAge()<<" " <<s[i].getCgpa()<<endl;
18     }
19
20     return 0;
21 }
```

Student (1/3):19, 3.9
Student (2/3):18, 3.75
Student (3/3):17, 3

Array of Objects as an Argument

- When we pass an array of objects to a function in C++, it is actually passed by reference.
- When we pass an array to a function, it decays into a pointer to its first element. This means that we are essentially passing the address of the first element of the array.
- When we pass a pointer to a function, we are passing the address of the variable holding the pointer, not a copy of the entire array. Therefore, any changes made to the array elements inside the function will affect the original array in the calling function.
- However, individual elements of an array are passed using pass-by-value mechanism.

Example: Passing Entire Array

```

1 class Test{
2     int x;
3 public:
4     Test(int t){x = t;}
5     int getX(){return x;}
6     void setX(int a){x = a;}
7 };
8 void f(Test ob[],const int s){
9     cout<<"Entering f():\n";
10    for(int i = 0; i<s; ++i)
11        cout<<ob[i].getX()<<" ";
12    cout<<endl;
13
14    for(int i = 0; i<s; ++i)
15        ob[i].setX(ob[i].getX()*2);
16    cout<<"Exiting f():\n";
17
18    for(int i = 0; i<s; ++i)
19        cout<<ob[i].getX()<<" ";
20    cout<<endl;
21 }

```

```

int main(){
    const int size = 3;
    Test ob[size] = {1,2,3};
    f(ob,size); //By Reference
    cout<<"Back in main():\n";
    for(int i = 0; i<3; ++i)
        cout<<ob[i].getX()<<" ";
    cout<<endl;

    return 0;
}

```

```

Entering f():
1 2 3
Exiting f():
2 4 6
Back in main():
2 4 6

```

Example: Passing Array Elements

```

1 class Test{
2     int x;
3     public:
4         Test(int t){x = t;}
5         int getX(){return x;}
6         void setX(int a){x = a;}
7 };
8 void f(Test x, Test y, Test z){
9     Test ob[] = {x,y,z};
10    cout<<"Entering f(): \n";
11    for(int i = 0; i<3; ++i)
12        cout<<ob[i].getX()<<" ";
13    cout<<endl;
14
15    for(int i = 0; i<3; ++i)
16        ob[i].setX(ob[i].getX()*2);
17
18    cout<<"Exiting f(): \n";
19    for(int i = 0; i<3; ++i)
20        cout<<ob[i].getX()<<" ";
21    cout<<endl;
22 }

```

```

int main(){

    Test ob[3] = {1,2,3};
    f(ob[0],ob[1],ob[2]);
    cout<<"Back in main():\n";
    for(int i = 0; i<3; ++i)
        cout<<ob[i].getX()<<" ";
    cout<<endl;

    return 0;
}

```

```

Entering f():
1 2 3
Exiting f():
2 4 6
Back in main():
1 2 3

```

Example

```
1 class Person {
2     int age;
3     public:
4         Person() {}
5         Person(int a) : age(a) {}
6         int getAge() { return age; }
7 };
8
9 void displayAges(Person arr[], int size) {
10     for (int i = 0; i < size; ++i) {
11         cout<<"Person " <<i + 1<<": Age = ";
12         cout<<arr[i].getAge()<<endl;
13     }
14 }
```

Example (Cont.)

```

1  int main() {
2      const int size = 4; int age;
3      Person persons[size];
4
5      for(int i = 0; i < size; ++i) {
6          cout<<"Enter age of Person("<<i + 1<<"/"<<size<<"): ";
7          cin >> age;  persons[i] = Person(age);
8      }
9
10     displayAges(persons, size);
11
12     return 0;
13 }
```

```

Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Person 1 Age: 19
Person 2 Age: 18
Person 3 Age: 17
Person 4 Age: 12
```


Exercise

Write a class *Student* with the following attributes and methods:

1 Attributes:

- name: A character array to store the name of the student.
- age: An integer variable to store the age of the student.

2 Methods:

- set(): Method to input the name and age of the student from user.
- display(): Method to display the name and age of the student.
- findMax(): Method to find the index of the oldest student among an array of *Student* objects.

3 The main() function should include the following functionalities:

- Create an array of *Student* objects to store the details of multiple students.
- Iterate through the array and input the name and age of each student using the set() method.
- Call the findMax() method to find the index of the oldest student among the students stored in the array.
- Display details of the oldest student using the display() method.

Solution

```

1  class Student {
2      char name[20]; int age;
3  public:
4      void set(){
5          cout <<"Enter your Name: "; cin >> name;
6          cout <<"Enter your age: "; cin >> age;
7      }
8      void display(){
9          cout<<"Name : "<<name<<endl;
10         cout<<"Age : "<<age<<endl;
11     }
12     int findMax(Student[], int);
13 };
14 int main() {
15     Student s[4], s1;
16     for(int i = 0; i<4; i++)
17         s[i].set();
18     int pos = s1.findMax(s, 4);
19     cout<<"\nOldest Student: "<<endl; s[pos].display();
20     return 0;
21 }

```

Enter your Name: Rayhaab
Enter your age: 19
Enter your Name: Muslim
Enter your age: 18
Enter your Name: Zakriya
Enter your age: 17
Enter your Name: Ayyat
Enter your age: 12

Oldest Student:
Name : Rayhaab
Age : 19

Solution (Cont.)

```
1 int Student::findMax(Student obj[], int size){  
2     int index = 0; int max = obj[0].age;  
3     for (int i = 1; i < size; i++) {  
4         if (obj[i].age > max) {  
5             max = obj[i].age;  
6             index = i;  
7         }  
8     }  
9     return index;  
10 }
```

Pointers to Objects

- To declare a pointer to an object of a particular class `className`, the following general form is used:

```
className* cptr;
```

- The pointer `cptr` can now holds the address of an object of type `className` using the following form:

```
cptr = &object;
```

- Before this statement to be executed, an object of type `className` must be declared using the following statement:

```
className object;
```

Pointers to Objects

- There are two ways to access the class members through pointers:
 - Using member selection operator (.)
 - Using pointer member access operator (->)
- First approach is a two-step process, in which we must first de-reference the pointer to obtain the object and then use this with the selection operator and the member name i.e.,

```
(*cptr).member;
```

Note: parentheses are necessary because member selection operator has higher precedence than that of indirection operator.

- In the second approach, the member access operator is formed using a minus sign and a greater than symbol.

```
cptr -> member;
```

Example 1

```
1 class Person{
2     int age;
3 public:
4     Person(int a){age = a;}
5     int getAge(){ return age;}
6 };
7
8 int main() {
9
10    Person obj(20), *p;
11    p = &obj; //get address of obj
12    cout<<p->getAge(); //use -> to call getAge()
13
14    return 0;
15 }
```

20

Example 2

```
1 class Person{
2     int age;
3 public:
4     Person(int a){age = a;}
5     int getAge(){ return age;}
6 };
7
8 int main() {
9
10    Person* p = new Person(20);
11    cout<<p->getAge();
12
13    delete p;
14
15    return 0;
16 }
```

20

Example: Passing Pointer to Object through a Function

```
1 class Person{
2     int age;
3 public:
4     Person(int a):age(a){}
5     int getAge(){return age;}
6     void setAge(int a){age = a;}
7 };
8
9 void byRef (Person* ob);
10
11 int main() {
12
13     Person obj(1974);
14     cout<<"Before Call: "<<obj.getAge()<<endl;
15     byRef(&obj);
16     cout<<"After Call: "<<obj.getAge()<<endl;
17
18     return 0;
19 }
20 void byRef(Person* ob){ob->setAge(2004);}
```

Before Call: 1974
After Call: 2004

Array of Pointers to Objects & Pointer to an Array of Objects

- An array of pointers to objects is different from a pointer to an array of objects.
- An array of pointers to objects stores multiple pointers, each pointing to a separate object.
- A pointer to an array of objects points to the entire array, treating it as a single entity.

Example 1: Array of Pointers to Objects

```
1 class Person {
2     int age;
3 public:
4     Person(){} //Required. Why?
5     Person(int a) : age(a) {}
6     int getAge(){return age;}
7 };
8 int main() {
9     const int size = 4; int age;
10    Person persons[size];
11    Person* array[size];
12    for(int i = 0; i < size; ++i){
13        cout<<"Enter age of Person("&<i+1<<"/"<<size<<"):";
14        cin>>age; persons[i] = Person(age);
15        array[i] = &persons[i];
16    }
17    for(int i = 0; i < size; ++i)
18        cout<<"Person "<<i+1<<": Age = ";
19        cout<<array[i]->getAge()<<endl;
20 return 0;
21 }
```

```
Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12
```

Example 2: Array of Pointers to Objects

```

1 class Person {
2     int age;
3 public:
4     Person(int a) : age(a) {}
5     int getAge(){return age;}
6 };
7 int main() {
8     const int size = 4; int age;
9     Person* array[size];
10
11     for(int i = 0; i < size; ++i){
12         cout<<"Enter age of Person("&<i+1<<"/"<<size<<"): ";
13         cin>>age; array[i] = new Person(age);
14     }
15     for(int i = 0; i < size; ++i){
16         cout<<"Person "<<i+1<<": Age = ";
17         cout<<array[i]->getAge()<<endl;
18     }
19     for(int i = 0; i < size; ++i) delete array[i];
20 return 0;
21 }

```

```

Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12

```

Example 1: Pointer to Array of Objects

```
1 class Person {
2     int age;
3 public:
4     Person(int a) : age(a) {}
5     int getAge() {return age;}
6 };
7
8 int main() {
9     const int size = 4;
10    Person persons[size] = {19, 18, 17, 12};
11    Person (*ptr)[size] = &persons; //Pointer to array
12
13    //Access using the pointer to the array
14    for(int i = 0; i < size; ++i) {
15        cout<<"Person " << i + 1 << ": Age = ";
16        cout<<(*ptr)[i].getAge()<<endl;
17    }
18    return 0;
19 }
```

Person 1 Age: 19
Person 2 Age: 18
Person 3 Age: 17
Person 4 Age: 12

Example 2: Pointer to Array of Objects

```

1 class Person {
2     int age;
3 public:
4     Person(){} //Required. Why?
5     Person(int a) : age(a) {}
6     int getAge(){return age;}
7 };
8 int main() {
9     const int size = 4; int age;
10    Person persons[size];
11    for(int i = 0; i < size; ++i){
12        cout<<"Enter age of Person("&<<i+1<<"/"<<size<<"):";
13        cin>>age; persons[i] = Person(age);
14    }
15    Person (*ptr)[size] = &persons;
16    for(int i = 0; i < size; ++i) {
17        cout <<"Person " <<i+1<<": Age = ";
18        cout<<(*ptr)[i].getAge()<<endl;
19    }
20    return 0;
21 }

```

```

Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Age of Person(1/4):19
Age of Person(2/4):18
Age of Person(3/4):17
Age of Person(4/4):12

```

Example: Passing Array of Pointers to Objects through a Function

```
1 class Person {
2     int age;
3
4 public:
5     Person() {}
6     Person(int a) : age(a) {}
7     int getAge() { return age; }
8 };
9
10 //Displays ages of persons pointed by the array of pointers
11 void displayAges(Person* arr[], int size) {
12     for (int i = 0; i < size; ++i) {
13         cout<<"Person " <<i + 1<<": Age = ";
14         cout<<arr[i]->getAge()<< endl;
15     }
16 }
```

Example (Cont.)

```

1  int main() {
2      const int size = 4;  int age;
3      Person* persons[size];
4
5      for (int i = 0; i < size; ++i) {
6          cout<<"Enter age of Person("&<<i + 1<<"/"<<size<<"):";
7          cin>>age;  persons[i] = new Person(age);
8      }
9
10     displayAges(persons, size);
11
12     for(int i = 0; i<size; ++i)
13         delete persons[i];
14
15     return 0;
16 }

```

```

Enter age of Person(1/4):19
Enter age of Person(2/4):18
Enter age of Person(3/4):17
Enter age of Person(4/4):12
Person 1 Age: 19
Person 2 Age: 18
Person 3 Age: 17
Person 4 Age: 12

```

Shallow Copy

- In shallow copy, an object is created by simply copying the data of all variables of the original object. This works well if none of the variables of the object are dynamically allocated otherwise, the copied object variable will also reference the same memory location. This will create ambiguity and run-time errors, dangling pointer.
- Since both objects will reference to the same memory location, then change made by one will also change the other object. If we want to create a replica of the object, this purpose will not be filled by the shallow copy.
- C++ compiler implicitly creates a copy constructor and overloads assignment operator in order to perform shallow copy at compile time.

Example 1: Copying Objects

```
1 class Copy{
2     int x;
3 public:
4     void set(int a){x = a;}
5     void show(){cout<<"x:"<<x;}
6 };
7
8 int main() {
9     cout<<"Object 1 ";
10    Copy c1;           //default constructor is called
11    c1.set(10);
12    cout<<"\n Object 2 -";
13    Copy c2 = c1;      //Copy through default Copy Constructor
14    c2.show();
15    cout<<"\n Object 3 -";
16    Copy c3;
17    c3 = c1;           //Copy through default assignment operator
18    c3.show();
19
20    return 0;
21 }
```

Object 1
Object 2 - x:10
Object 3 - x:10

Example 2: Copying Objects

```
1 class Copy{
2     int x;
3     public:
4     Copy(){cout<<"Constructor";}
5     Copy(const Copy &c){cout<<"Copy"; x = c.x;} //this->x = c.x;
6     Copy operator=(const Copy c)
7         {cout<<"\n Assignment";x = c.x;return *this;} //this->x=c.x;
8     void set(int a){x = a;}    void show(){cout<<"x:"<<x;}
9 };
10 int main() {
11     cout<<"Object 1 ";
12     Copy c1;           //Constructor is called
13     c1.set(10);
14     cout<<"\n Object 2 -";
15     Copy c2 = c1;      //Copy through Copy Constructor:  c2(c1)
16     c2.show();
17     cout<<"\n Object 3 -";
18     Copy c3;
19     c3 = c1;           //Copy through assignment operator:  c3(c1)
20     c3.show();
21     return 0;
22 }
```

Object 1 - Constructor
Object 2 - Copy x:10
Object 3 - Constructor Copy
Assignment Copy x:10

Example 3: Copying Objects

```

1  class Copy{
2      int x;
3  public:
4      Copy(){cout<<"Constructor";}
5      Copy(const Copy &c){cout<<"Copy"; x = c.x;} //this->x = c.x;
6      Copy& operator=(const Copy& c)
7          {cout<<"\n Assignment"; x = c.x; return *this;} //this->x=c.x;
8      void set(int a){x = a;}    void show(){cout<<"x:"<<x;}
9  };
10 int main() {
11     cout<<"Object 1 ";
12     Copy c1;           //Constructor is called
13     c1.set(10);
14     cout<<"\n Object 2 -";
15     Copy c2 = c1;      //Copy through Copy Constructor:  c2(c1)
16     c2.show();
17     cout<<"\n Object 3 -";
18     Copy c3;
19     c3 = c1;           //Copy through assignment operator:  c3(c1)
20     c3.show();
21     return 0;
22 }

```

Object 1 - Constructor
Object 2 - Copy x:10
Object 3 - Constructor
Assignment x:10

Example 4: Copying Objects

```
1 class Copy{
2     int x;
3 public:
4     void set(int a){x = a;}
5     void show(){cout<<" x:"<<x;}
6 };
7
8 int main() {
9
10    cout<<" Object 1 -";
11    Copy c1;
12    c1.set(10);
13    cout<<"\n Object 2 -";
14    Copy c2(c1);    //Copy through default Copy Constructor
15    c2.show();
16
17    return 0;
18 }
```

Object 1 -
Object 2 - x:10

Example 5: Copying Objects

```
1 class Copy{
2     int x;
3     public:
4         Copy(){cout<<"Constructor ";}
5         Copy(const Copy &c){cout<<"Copy"; this->x = c.x;}
6         void set(int a){x = a;}
7         void show(){cout<<" x:"<<x;}
8 };
9
10 int main() {
11
12     cout<<" Object 1 -";
13     Copy c1;
14     c1.set(10);
15     cout<<"\n Object 2 -";
16     Copy c2(c1);    //Copy through Copy Constructor
17     c2.show();
18
19     return 0;
20 }
```

Object 1 - Constructor
Object 2 - Copy x:10

Example 6: Copying Objects

Object 1 -
Object 2 - x :10, *p :5 x :10, *p :5

```
1 class Copy{
2     int x;
3     int *p;
4 public:
5     void set(int a, int b){
6         x = a;
7         p = &b; //Each object has its own p pointing to its own location b
8     } //b does not exist here
9     void show(){cout<<" x :"<<x<<" , *p :"<<*p;}
10 };
11 int main() {
12     cout<<" Object 1 -"; Copy c1; int z = 5;
13     c1.set(10, z);
14     cout<<"\n Object 2 -";
15     {
16         Copy c2 = c1; //Copy through default Copy Constructor
17         c2.show(); //Note: c2 will destroy after this block
18     }
19     c1.show();
20     return 0;
21 }
```

Example 7: Copying Objects

```
1 class Copy {
2     int* data;
3 public:
4     Copy(int val) {data = &val;}
5
6     // Copy constructor performing shallow copy
7     Copy(const Copy& other) {
8         data = other.data; //Shallow copy of the pointer
9     }
10
11     void displayData()
12         {cout<<"Value: "<<*data<<" at address:"<<data<<endl;}
13
14     void setData(int val) {*data = val;}
15 };
```

Example 7: Copying Objects (Cont.)

```
1 int main() {  
2  
3     int value = 5;  
4     Copy obj1(value);  
5  
6     cout << "Original ";  
7     obj1.displayData();  
8     Copy obj2 = obj1; //shallow copy using copy constructor  
9     cout << "Copied ";  
10    obj2.displayData();  
11  
12    obj1.setData(10);  
13  
14    cout << "After modification in original object:\n";  
15    obj1.displayData();  
16  
17    cout << "Copied object after modifying original object:\n";  
18    obj2.displayData();  
19  
20    return 0;  
21 }
```

Original Value: 5 at address: 0x916c80
Copied Value: 5 at address: 0x916c80
After modification in original object:
Value: 10 at address: 0x916c80
Copied object after modifying original object:
Value: 10 at address: 0x916c80

Example 8: Copying Objects

Object 1 -
Object 2 - x :10, *p :5 x :10, *p :12802416

```
1 class Copy{
2     int x;
3     int *p;
4 public:
5     Copy(){p = new int;}
6     void set(int a, int b){x = a; *p = b;}
7     void show(){cout<<" x : "<<x<<" , *p : "<<*p;}
8     ~Copy(){delete p;}
9 };
10
11 int main() {
12     cout<<" Object 1 -"; Copy c1; int z = 5;
13     c1.set(10, z);
14     cout<<"\n Object 2 -";
15     {
16         Copy c2 = c1; //Copy through default Copy Constructor
17         c2.show(); //c2 exits in this block only
18     } //As c2 destroys, the shared location pointed to by p also destroys
19     c1.show();
20     return 0;
21 }
```

Example 9: Copying Objects

```
1 class ShallowCopy {
2     int* data;
3 public:
4     ShallowCopy(int val) {data = new int(val);}
5
6     // Copy constructor performing shallow copy
7     ShallowCopy(const ShallowCopy& other) {
8         data = other.data; //Shallow copy of the pointer
9     }
10
11     void displayData()
12         {cout<<"Value: " <<*data<<" at address:"<<data<<endl;}
13
14     void setData(int val) {*data = val;}
15     ~ShallowCopy() {delete data;}
16 };
```

Example 9: Copying Objects (Cont.)

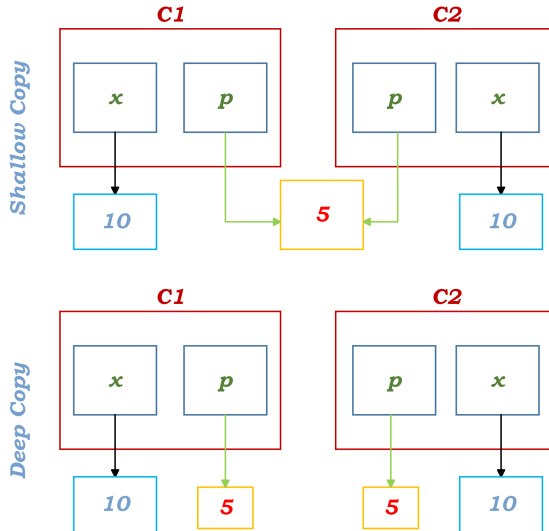
```
1 int main() {  
2  
3     ShallowCopy obj1(5);  
4  
5     cout << "Original ";  
6     obj1.displayData();  
7     ShallowCopy obj2 = obj1; //shallow copy using copy constructor  
8     cout << "Copied ";  
9     obj2.displayData();  
10  
11    obj1.setData(10);  
12  
13    cout<<"After modification in original object:\n";  
14    obj1.displayData();  
15  
16    cout<<"Copied object after modifying original object:\n";  
17    obj2.displayData();  
18  
19    return 0;  
20 }
```

Original Value: 5 at address: 0x916c80
Copied Value: 5 at address: 0x916c80
After modification in original object:
Value: 10 at address: 0x916c80
Copied object after modifying original object:
Value: 10 at address: 0x916c80

Deep Copy

- In deep copy, an object is created by copying data of all variables, and it also allocates similar memory resources with the same value to the object.
- In order to perform deep copy, we need to explicitly define the copy constructor and assign dynamic memory as well, if required. Also, it is required to dynamically allocate memory to the variables in the other constructors, as well.
- Since both objects will reference to the different memory locations, then change made by one will not affect the other object. If we want to create a replica of the object, this purpose the deep copy will serve the purpose.

Deep Copy & Shallow Copy



Example 1: Copying Objects

Object 1 - Constructor
Object 2 - Copy x :10, *p :5 x :10, *p :5

```
1 class Copy{
2     int x;
3     int *p;
4 public:
5     Copy(){cout<<"Constructor "; p = new int;}
6     Copy(const Copy &c)
7         {cout<<"Copy "; x = c.x; p = new int(*(c.p));}
8     void set(int a, int b){x = a; *p = b;}
9     void show(){cout<<" x : "<<x<<" , *p : "<<*p;}
10    ~Copy(){delete p;}
11 };
12 int main() {
13     cout<<" Object 1 -"; Copy c1; int z = 5; c1.set(10, z);
14     cout<<"\n Object 2 -";
15     {
16         Copy c2 = c1; //Copy through Copy Constructor
17         c2.show(); //c2 exists in this block only
18     } //As c2 destroys, the location pointed to by p for c1 still exists
19     c1.show();
20     return 0;
21 }
```

Example 2: Copying Objects

```
1 class Copy {
2     int* data;
3 public:
4     Copy(int val) { data = &val; }
5
6     Copy(const Copy& other) {
7         data = new int(*(other.data)); //Deep copy of the pointer
8     }
9
10    void displayData() {
11        cout<<"Value: "<<*data<<" at address: ";
12        cout<<data<<endl;
13    }
14
15    void setData(int val) {*data = val;}
16    //No need to delete 'data' as it points to an external variable
17    ~Copy() {delete data;}
18 };
```

Example 2: Copying Objects (Cont.)

```
1 int main() {
2     int value = 5;
3     Copy obj1(value);
4
5     cout<<"Original ";
6     obj1.displayData();
7
8     Copy obj2 = obj1; //Deep Copy using copy constructor
9     cout<<"Copied ";
10    obj2.displayData();
11
12    obj1.setData(10);
13
14    cout<<"After modification in original object:\n";
15    obj1.displayData();
16
17    cout<<"Copied object after modifying original object:\n";
18    obj2.displayData();
19    return 0;
20 }
```

Original Value: 5 at address: 0x916c80
Copied Value: 5 at address: 0x916c80
After modification in original object:
Value: 10 at address: 0x916c80
Copied object after modifying original object:
Value: 5 at address: 0x916c80

Example 3: Copying Objects

```
1 class DeepCopy {
2     int* data;
3 public:
4     DeepCopy(int val) {data = new int(val);}
5
6     DeepCopy(const DeepCopy& other) {
7         data = new int(*(other.data)); //Deep copy of pointed value
8     }
9
10    void displayData() {
11        cout<<"Value: "<<*data<<" at address: ";
12        cout<<data<<endl;
13    }
14
15    void setData(int val) {*data = val;}
16    //No need to delete 'data' as it points to an external variable
17    ~DeepCopy() {delete data;}
18 };
```

Example 3: Copying Objects (Cont.)

```
1 int main() {  
2     DeepCopy obj1(5);  
3  
4     cout<<"Original ";  
5     obj1.displayData();  
6  
7     DeepCopy obj2 = obj1; //Deep Copy using copy constructor  
8     cout<<"Copied ";  
9     obj2.displayData();  
10  
11    obj1.setData(10);  
12  
13    cout<<"After modification in original object:\n";  
14    obj1.displayData();  
15  
16    cout<<"Copied object after modifying original object:\n";  
17    obj2.displayData();  
18    return 0;  
19 }
```

Original Value: 5 at address: 0x916c80
Copied Value: 5 at address: 0x916c80
After modification in original object:
Value: 10 at address: 0x916c80
Copied object after modifying original object:
Value: 5 at address: 0x916c80